Project 4 Report
Course: EECE 5614
Student: Caleb Bloch
Professor: Mahdi Imani
Semester: Spring 2026
Date: 4/20/2026

Abstract:

Goal:
The goal for this project is to learn a control policy for a stochastic maze using Deep Q-Network methods. I first implemented standard DQN with replay memory, a target network, minibatch training, and an epsilon-greedy exploration schedule. I then compared the learned results under different learning rates, and finally implemented Double DQN and Dueling DQN to compare their training behavior, final policy, value estimates, and sampled start-to-goal path against the standard DQN baseline.

Methods:
I represented each maze state as a normalized coordinate vector s = [x, y]^T, where x and y are scaled to the range [0, 1]. The maze transitions followed the project specification: with probability 1 - P the agent moved in the intended direction, and with probability P/2 each it moved to one of the two perpendicular neighboring cells, using P = 0.025. The reward function used -1 for taking an action, an additional -5 for yellow cells, an additional -10 for red cells, an additional -0.8 for hitting a wall, and +100 for reaching the goal. Episodes started from a random initial non-goal state and ended when the goal was reached or the maximum episode length was hit.

Results:
The standard DQN learned a successful policy that reached the goal from the blue start state with a sampled path length of 12. Its final moving-average reward was 86.08 and its final moving-average episode length was 7.56, which indicates that the network learned a short and efficient route to the goal. In the learning-rate experiment, alpha = 0.0001 performed much worse than the baseline, while alpha = 0.01 was slightly better than alpha = 0.001, suggesting that the smaller learning rate was too slow while the larger one was still stable and effective. Double DQN slightly outperformed standard DQN, reaching final average reward 88.112 and lower average episode length 6.84. Dueling DQN also performed strongly, with final average reward 87.624 and noticeably lower final average loss than the standard baseline. Overall, all three methods learned a valid path, but Double DQN gave the strongest final maze performance in this implementation.

Problem Setup:

Maze Environment:
This project uses an 8 x 8 stochastic maze with empty cells, wall cells, yellow penalty cells, red penalty cells, one blue start cell, and one green goal cell. The agent can choose one of four actions: Up, Right, Down, or Left. Because the environment is stochastic, selecting one action

does not always move the agent exactly in that direction. If the chosen or resulting move would hit a wall, the agent stays in its current state.

State Representation:
I represented each state by its (x, y) coordinate location in the maze. To make the network input simple and numerically stable, I normalized both coordinates to [0, 1] before passing them to the neural network. So the first input dimension corresponds to the horizontal position x, and the second input dimension corresponds to the vertical position y.

Objective:
The objective is to learn a policy that reaches the goal state with high accumulated reward while avoiding unnecessary delay and hazardous states. Since each step has a negative reward, a good policy should also find a relatively short path to the goal instead of wandering through the maze.

Evaluation:
I evaluated the learned agent using the moving-average reward, moving-average loss, and moving-average episode length over training. I also inspected the final learned policy, the final state values obtained by taking the maximum Q-value at each state, and a sampled path from the blue start state following the final learned policy. For the comparison experiments, I used these same outputs to compare standard DQN against Double DQN and Dueling DQN.

Methods:

Network Structure
For the standard DQN, I used the same forward feeding network structure suggested in the handout: an input layer of size 2, three hidden layers of size 128 with traditional ReLU activations, and an output layer of size 4 corresponding to the Q-values of the four actions. For Dueling DQN, I used a shared feature extractor followed by a value stream and an advantage stream, and combined them using the dueling-network equation from the project statement.

Training Process
I used Adam for learning rate optimization. The target network was updated gradually using the soft update rule after each training step. The epsilon schedule was epsilon_k = max(0.1, 0.995^k), so the agent explored more in the early episodes and then gradually shifted toward exploitation. The replay memory stored transitions in the form (s, a, r, s', done). Once a warmup amount of experience had been collected, the Q-network was updated by randomly sampling minibatches from the memory and minimizing the mean-squared Bellman error. The full set of parameters and hyperparameters used in the baseline standard DQN is reported in Part 1 below.

Standard DQN
In the standard DQN implementation, the target value was computed using the maximum target-network Q-value at the next state. Once a warmup amount of experience was collected, the Q-network was updated by randomly sampling minibatches from the memory and minimizing the mean-squared Bellman error.

Double DQN

For Double DQN, I kept the same training loop and network structure but changed the target computation. Instead of using the target network to both choose and evaluate the next action, I selected the next action using the online Q-network and then evaluated that selected action using the target network. This is the main change required to reduce overestimation bias.

Dueling DQN
For Dueling DQN, I changed the network architecture so that after the shared feature layers, the model branched into a value stream V(s) and an advantage stream A(s, a). These two outputs were then combined into Q-values using Q(s, a) = V(s) + (A(s, a) - mean_a' A(s, a')). The loss function and training loop remained the same as in standard DQN.

Results:

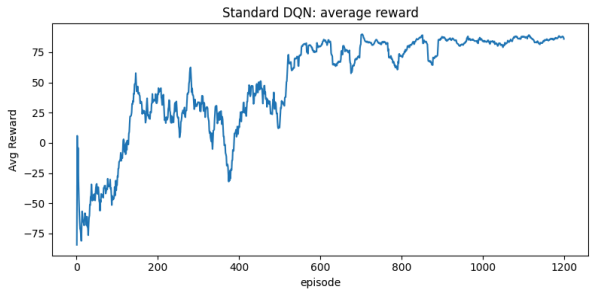
Part 1: Parameters and Hyperparameters
The final parameters and hyperparameters used in the baseline standard DQN implementation were:

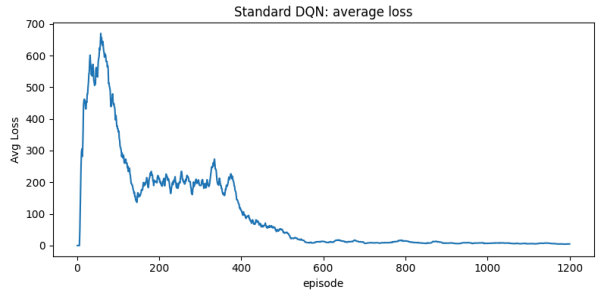
- Replay memory size = 10000
- Maximum episode length T_epi = 50
- Q-network update frequency N_QU = 4
- Discount factor gamma = 0.99
- Minibatch size N_batch = 64
- Learning rate alpha = 0.001
- Soft update hyperparameter eta = 0.01
- Number of episodes N_epi = 1200
- Epsilon schedule = max(0.1, 0.995^k)
- State representation = normalized (x, y) in [0, 1]

These values worked well in my implementation and produced a stable standard DQN baseline that learned a valid path to the goal.

Part 2: Standard DQN reward and loss

- Final moving-average reward = 86.08
- Final moving-average loss = 5.1461

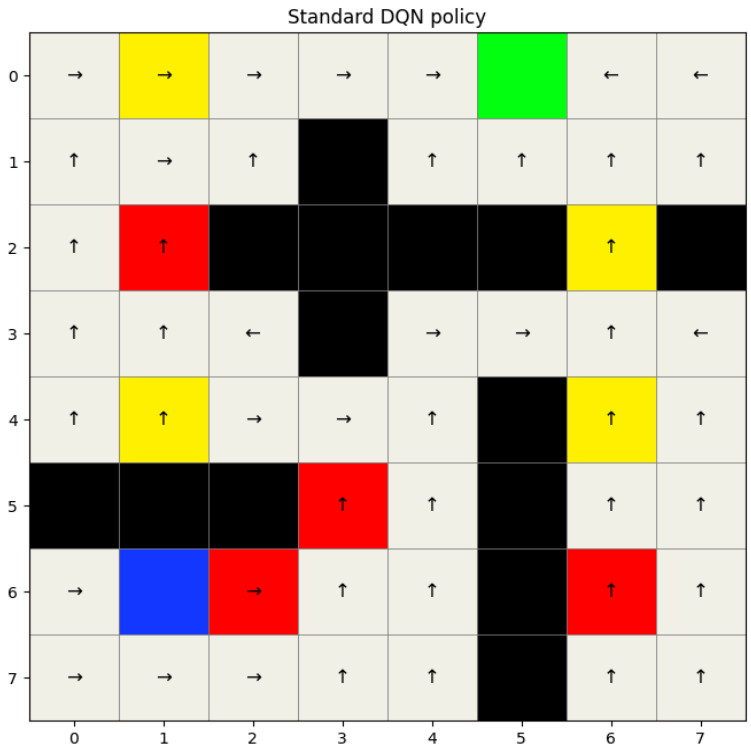




For the baseline DQN, the moving-average reward increased during training and finished at 86.08, while the moving-average loss decreased and stabilized around 5.1461. This is the trend I would expect: as the network learns better Q-values, the policy reaches the goal more

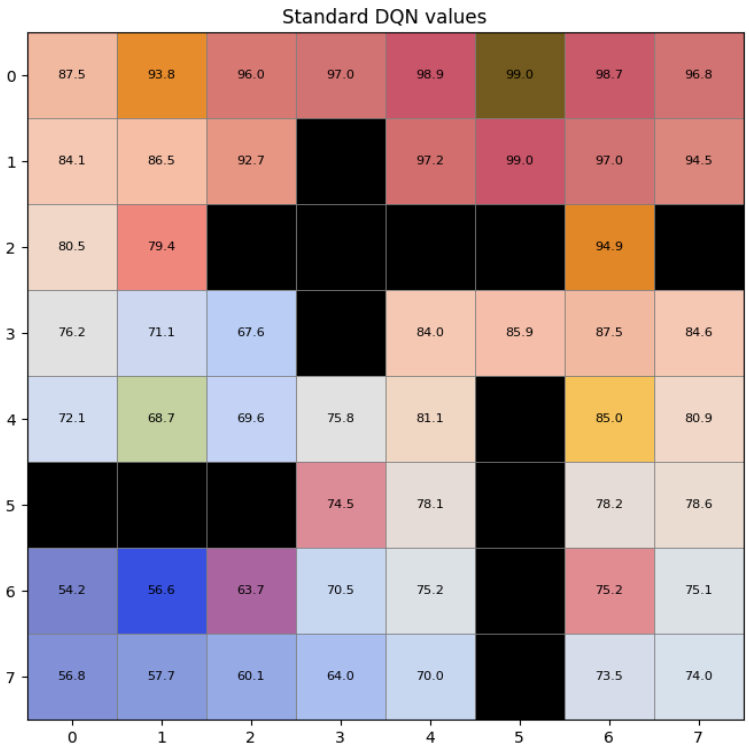
efficiently and the Bellman targets become more consistent, so reward improves and loss becomes more stable. There are still fluctuations because the environment is stochastic and the updates are based on replayed samples, but the final trend is clearly favorable.

Part 3: Final obtained policy



The final learned policy is intuitively close to optimal. It directs the agent toward the goal region while avoiding the worst hazard cells and walls whenever possible. Since the policy successfully leads from the blue start state to the goal in a short sampled path, it appears to have captured the main structure of the maze rather than simply memorizing random local moves.

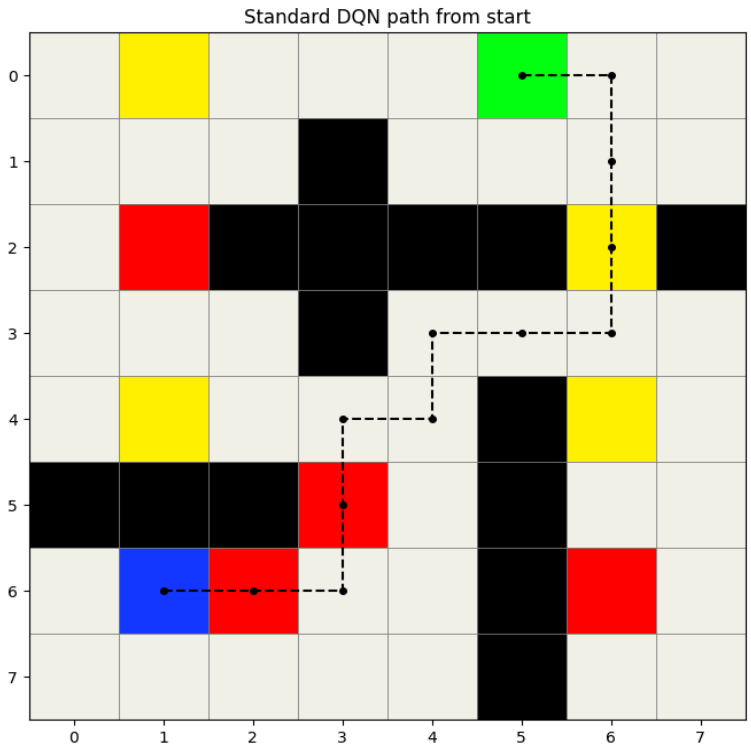
Part 4: Final state values



The final state values are given by the maximum Q-value at each state. The numerical range is what I would expect: values are larger in states that are closer to the goal or that have good access to the goal, and smaller in states near walls, red cells, yellow cells, or longer detours. Because the goal reward is +100 and the agent still pays step costs and hazard costs before reaching it, the values should remain positive in many good states but not reach unrealistically large numbers. The learned values in this implementation follow that pattern.

Part 5: Path from the start state

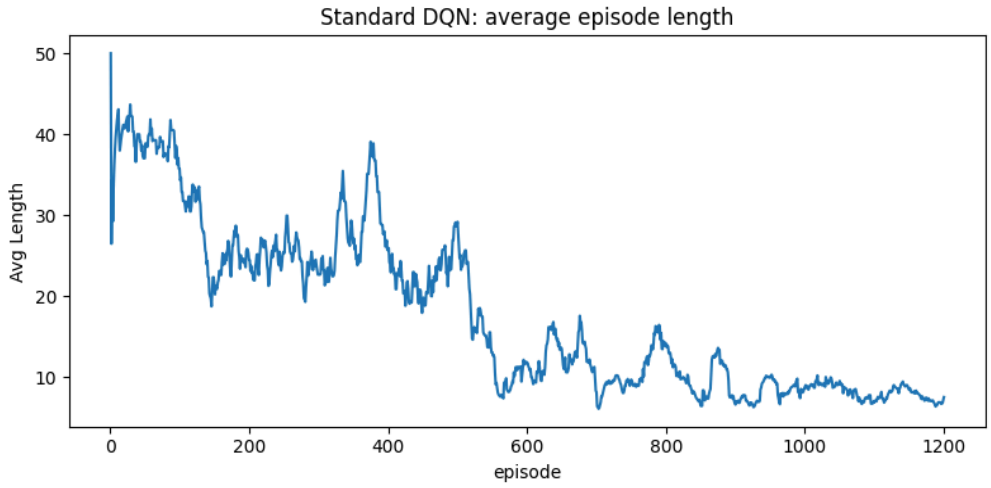
- Sampled path length = 12



Starting from the blue start state and following the final learned standard DQN policy, the sampled path reaches the goal successfully. The sampled path length is 12, which is pretty good relative to the maze size.

Part 6: Average episode length

- Final moving-average episode length = 7.56



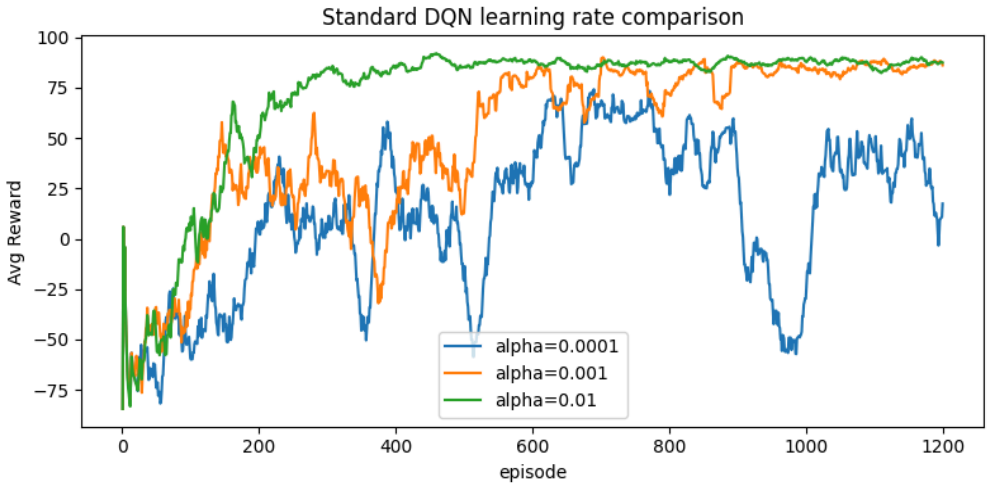
The final moving-average episode length for the standard DQN is 7.56. This is closely related to the reward curve from Part 2. As the agent learns a better policy, it reaches the goal in fewer

steps, so the episode length drops. Since each step carries a penalty of -1, shorter episodes also produce a larger reward. So the decrease in Avg Length is consistent with the increase in Avg Reward.

Part 7: Effect of the learning rate

I compared the baseline learning rate alpha = 0.001 with a much smaller learning rate alpha = 0.0001 and a much larger learning rate alpha = 0.01. The final moving-average rewards were:

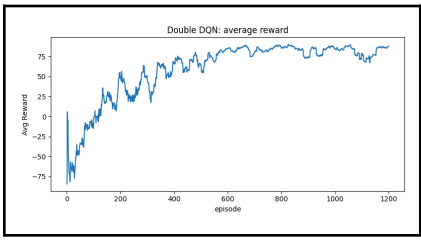
- alpha = 0.0001: Avg Reward = 17.4000
- alpha = 0.001: Avg Reward = 86.0800
- alpha = 0.01: Avg Reward = 87.4000

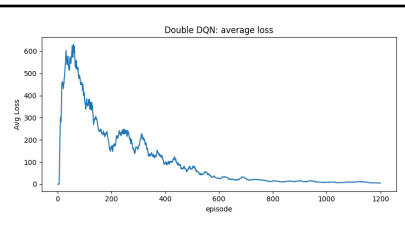


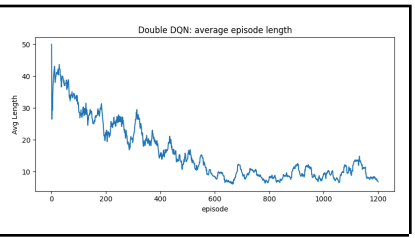
The much smaller learning rate is clearly too slow. Even after 1200 episodes, the learned performance remains far below the other two settings. The larger learning rate of 0.01 is still reasonable in this problem and actually gives slightly better final reward than the baseline. In this experiment, the baseline and larger learning rates both work well, while the very small learning rate prevents the network from improving fast enough.

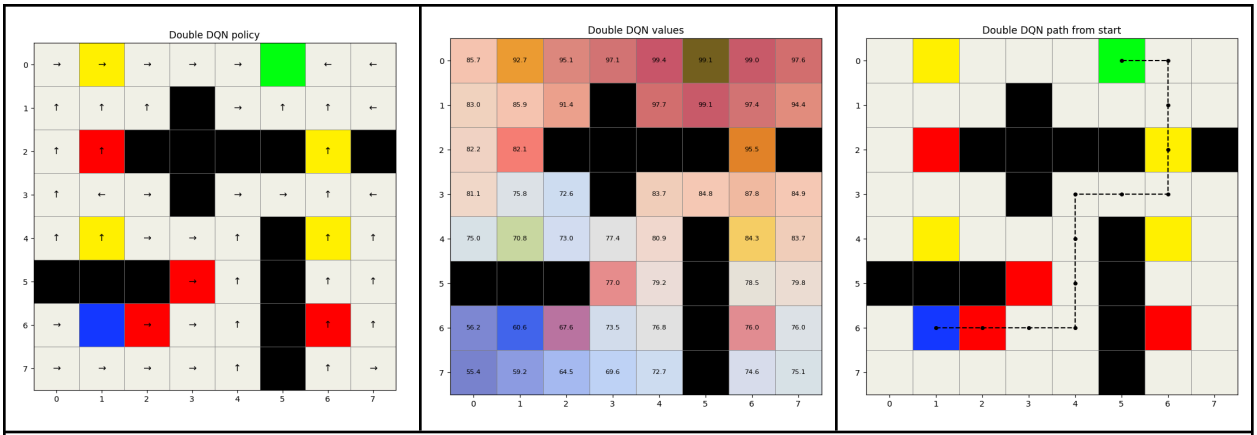
Part 8: Double DQN

- Final moving-average reward = 88.1120
- Final moving-average loss = 4.8963
- Final moving-average episode length = 6.84
- Sampled path length from blue start = 12





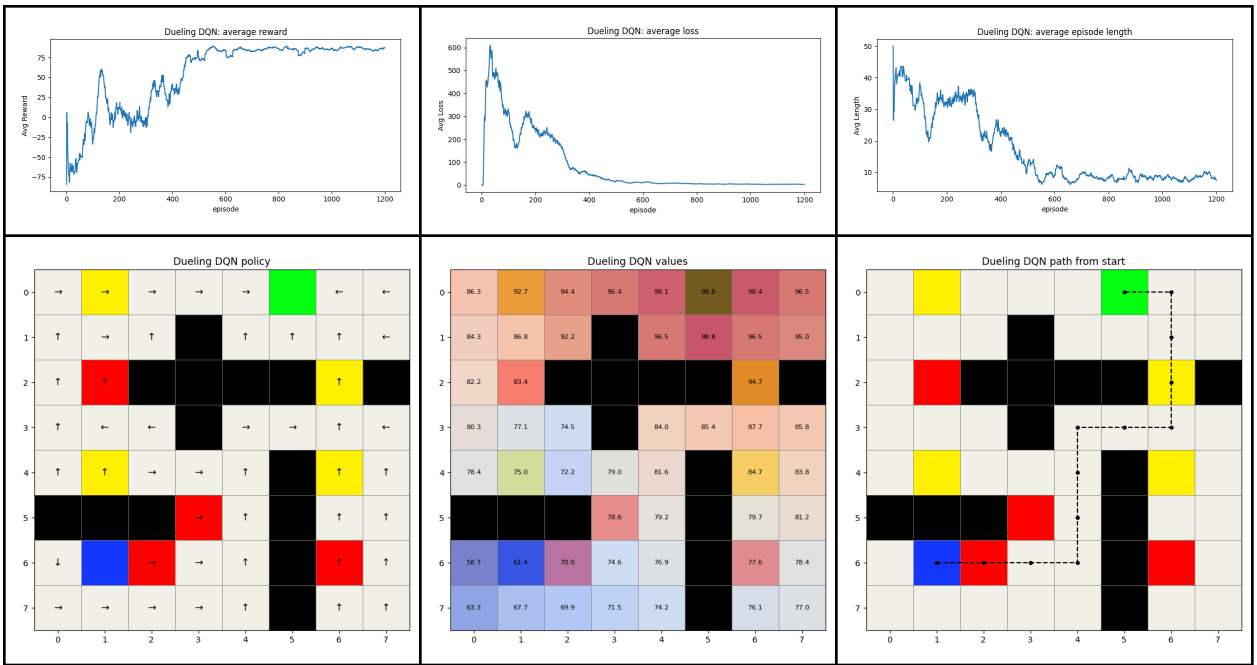




Comparison with standard DQN: Double DQN performs slightly better than standard DQN in this experiment. It achieves a somewhat larger final reward and a lower final episode length, which suggests that the learned policy is slightly more efficient. The loss is also a little lower. This matches the usual intuition behind Double DQN: separating action selection from action evaluation reduces overestimation bias and can lead to more stable learning.

Part 9: Dueling DQN

- Final moving-average reward = 87.6240
- Final moving-average loss = 3.6085
- Final moving-average episode length = 7.52
- Sampled path length from blue start = 12



Comparison with standard DQN and Double DQN: Dueling DQN also performs well and clearly learns a successful policy. Its final reward is slightly better than standard DQN but slightly below Double DQN in this implementation. At the same time, its final loss is

noticeably smaller than both the standard and Double DQN baselines, which suggests that the value/advantage decomposition helps the network organize the learning problem more cleanly. Even though Double DQN gives the best final reward here, Dueling DQN is still competitive and stable.

Overall Takeaway:
Across all experiments, the DQN methods successfully learned good policies for the subject maze. Standard DQN already gives a strong baseline and reaches the goal from the blue start state using a short path. The learning rate experiment shows that too small a learning rate can severely slow learning, while a somewhat larger learning rate can still work well. Double DQN gives the best overall performance in my implementation, with the highest final reward and shortest final average episode length. Dueling DQN also performs strongly and achieves the smallest final loss. Overall, the findings confirms that DQN and its variations can learn effective policies even in a stochastic maze when replay memory, target networks, and a reasonable epsilon schedule are used.

AI Use / Acknowledgment:
I used ChatGPT for conceptual clarification and minor debugging/formatting help while implementing the DQN simulations and organizing the write-up. All final code and documentation was written, reviewed, and executed by me.

Citation: OpenAI, *ChatGPT*, accessed April 2026.